

Updated Autonomous Intersection Concept

Brief alignment note for requirements, UML diagrams, UPPAAL, and HW/SW co-design

Core concept

We will model the intersection as a **reservation-based critical-zone system**. The intersection is divided into critical zones, and each critical zone is treated as a shared resource. Vehicles are scheduled along a timeline and may only enter a critical zone during their assigned time slot.

Main safety rule: two vehicles may not reserve or occupy the same critical zone at the same time. Multiple vehicles may cross simultaneously if their critical-zone reservations do not conflict.

Updated assumptions and limits

- Directions are named **DN, DE, DS, DW** for vehicles coming from North, East, South, and West.
- Critical zones are named **CZ0, CZ1, CZ2, CZ3** using matrix order: top-left to bottom-right.
- Each vehicle gets a unique ID and timestamp when it enters the scheduling range.
- Each vehicle declares an origin and destination, for example **DN -> DS**.
- A lookup table maps each origin/destination pair to its required critical-zone sequence.
- Each direction has a FIFO queue. Vehicles in the same lane may not overtake.
- Baseline model: no speed-up. Vehicles may maintain speed or slow down only.
- Minimum speed is a design parameter that we will test experimentally.
- If no valid reservation exists inside the feasible arrival window, the vehicle is marked as unable to schedule and the failure reason is recorded.
- Slack time is used for urgency. Smaller slack means the vehicle is closer to missing its latest feasible arrival time.
- V2I/controller scheduling is the main control path. V2V is only redundancy/awareness, not the main scheduler.

Main scheduling flow

1	Vehicle appears in scheduling range
2	Assign ID and timestamp
3	Vehicle declares origin and destination
4	Controller uses LUT to get critical-zone sequence
5	Controller calculates feasible arrival window from starting speed and minimum speed
6	Controller searches reservation timeline for the earliest non-conflicting sequence
7	If found: reserve time slots and send schedule to vehicle
8	If not found: mark vehicle as unable to schedule and record the reason

What to update in our diagrams

- **Requirements:** add reservation timeline, critical-zone time slots, FIFO lane order, no overtaking, minimum speed, slack time, scheduling failure reasons, and HW conflict checking.
- **Use cases:** detect/generate vehicle, assign ID/timestamp, map route to critical zones, search timeline, reserve slots, or mark unable to schedule.
- **Activity diagram:** show the flow from vehicle generation to reservation success/failure.
- **Sequence diagram:** show Vehicle -> Controller -> Reservation Timeline -> Conflict Checker -> Vehicle.
- **UPPAAL:** verify no deadlock, no two vehicles in the same critical zone, valid reservation before entry, and FIFO order per direction.

- **HW/SW co-design:** software handles scheduling; hardware/VHDL handles critical-zone conflict checking and reservation-validity checks.

Frozen design statement

The baseline system schedules each vehicle immediately when it enters range. Vehicles have random or defined origin/destination pairs, are mapped to critical-zone sequences using a lookup table, and are assigned the earliest non-conflicting reservation within their feasible arrival window. The baseline does not allow overtaking or speed-up. If the vehicle cannot be scheduled, the failure is recorded so we can evaluate system limits and later test improvements such as different minimum speeds or detection distances.